SDEV2150

Lesson 23: Shared Client State

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Lesson Objectives

In this lesson, students will:

- Explain the difference between local and shared state
- Identify when state needs to be shared across components
- Understand how the Context API works
- Use a provider to share state
- Consume shared state in multiple components

WE ARE ESSENTIAL TO ALBERTA
NAIT

Agenda

- Local state review
- The problem of prop drilling
- What is Context?
- Creating and providing context
- Consuming context
- When to use Context
- When not to use Context

WE ARE ESSENTIAL TO ALBERTA
NAIT

CONNECTING

Where should state live?

From previous lessons:

- rendering can happen on the client or server
- data can come from different sources

Now the question becomes:

Where should application state live?

WE ARE ESSENTIAL TO ALBERTA | NAIT

Types of state

- Local component state
- Shared client state
- Server state (coming next lesson)

WE ARE ESSENTIAL TO ALBERTA
NAIT

LOCAL STATE

Example

```
const [count, setCount] = useState(0);
```

WE ARE ESSENTIAL TO ALBERTA
NAIT

When local state works well

- Used by a single component
- Not needed elsewhere
- Simple and isolated

Limitation

- What if multiple components need the same state?

PROP DRILLING

WE ARE ESSENTIAL TO ALBERTA
NAIT

The problem

Passing state through components that don’t use it:

```
App -> Layout -> Sidebar -> Button
```

WE ARE ESSENTIAL TO ALBERTA
NAIT

Issues

- Hard to maintain
- Tight coupling between components
- Scales poorly

WE ARE ESSENTIAL TO ALBERTA
NAIT

Navigation in CSR

After the first load:

- navigation is handled by the client
- no full page reload
- only data may be fetched

This creates a smoother user experience.

WE ARE ESSENTIAL TO ALBERTA
NAIT

INTRODUCING CONTEXT

What is Context?

Context allows us to:

- share state across the component tree
- avoid passing props through every level

WE ARE ESSENTIAL TO ALBERTA
NAIT

Key idea

Context provides shared client state to any component that needs it.

CREATING CONTEXT

Step 1: Create a context

```
import { createContext } from 'react';

export const ThemeContext = createContext();
```

WE ARE ESSENTIAL TO ALBERTA
NAIT

Step 2: Create a provider

```
function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');

  function toggleTheme() {
    setTheme(prev => prev === 'light' ? 'dark' : 'light');
  }

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}
```

WE ARE ESSENTIAL TO ALBERTA
NAIT

PROVIDING CONTEXT

Wrap the application

```
<ThemeProvider>
  <App />
</ThemeProvider>
```

WE ARE ESSENTIAL TO ALBERTA
NAIT

Result

- All components inside can access the shared state.

CONSUMING CONTEXT

Using useContext

```
const { theme, toggleTheme } = useContext(ThemeContext);
```

WE ARE ESSENTIAL TO ALBERTA
NAIT

Example

```
<button onClick={toggleTheme}>
  Current theme: {theme}
</button>
```

WE ARE ESSENTIAL TO ALBERTA
NAIT

SHARED STATE IN ACTION

Key behaviour

- Multiple components use the same state
- Updating in one place updates everywhere

WE ARE ESSENTIAL TO ALBERTA
NAIT

Example locations

- Header
- Sidebar
- Page content

WHEN TO USE CONTEXT

Good use cases

- Theme / UI preferences
- Authenticated user
- Shared UI state

WHEN NOT TO USE CONTEXT

Avoid Context for

- Frequently changing data
- Complex state logic
- Server-fetched data

WE ARE ESSENTIAL TO ALBERTA
NAIT

Why?

Context does not handle:

- caching
- async state
- background updates

WE ARE ESSENTIAL TO ALBERTA
NAIT

CLIENT VS SERVER STATE

WE ARE ESSENTIAL TO ALBERTA
NAIT

Client state

- UI-related
- local or shared

Server state

- comes from APIs
- changes over time
- needs synchronization

WE ARE ESSENTIAL TO ALBERTA
NAIT

Key idea

Different types of state require different tools.

HANDS-ON PRACTICE

Exercise: Update the existing walkthrough project to use context for client side state management.

Objective: Implement client-side state management

- Obtain the exercise starter files
  1. Create the necessary context and provider files.
  2. Configure the provider for the app.
  3. Utilize the context in components, where necessary.
  4. Ask questions if you are unsure of what to do or how the code works.

WE ARE ESSENTIAL TO ALBERTA | NAIT

QUICK POLL

When should you use Context?

A. For API data fetching
B. For shared UI state
C. For caching server data
D. For all application state

WE ARE ESSENTIAL TO ALBERTA
NAIT

Lesson Summary

- In this lesson, we covered the following:

    1. Explain the difference between local and shared state
    2. Identify when state needs to be shared across components
    3. Understand how the Context API works
    4. Use a provider to share state
    5. Consume shared state in multiple components

- Complete the lesson 23 checklist

WE ARE ESSENTIAL TO ALBERTA | NAIT

EXIT TICKET

Explain:

1. What Context is
2. When to use Context
3. One limitation of Context
4. How Context differs from local state

WE ARE ESSENTIAL TO ALBERTA
NAIT